



PLATFORM ENGINEERS • DEPLOYERS

DEVOPS MANUAL

Local dev, build/serve, Cloudflare, VPS, CORS, CI

Open charting PWA — own the axes, swap the engine



CONTENTS

01 DevOps

02 Local development

03 Build and serve

04 Cloudflare deployment

05 VPS demo topology

06 CORS and origins

07 CI and testing



DEVOPS

Build, serve, Cloudflare, VPS demo, CORS, and CI for the AXIS PWA and Worker.

DEVOPS

Abstract

AXIS DevOps spans three runnable surfaces:

- 1. **Vite + Solid PWA** (`frontend/`) — product UI
- 2. **Static server** (`axis_pwa_server.py` or Vite preview) — production-like assets
- 3. **Cloudflare Worker** (`frontend/worker/`) — edge API / WS

Python evaluation for desk workflows still often uses the **Flask Pro API** (`make run` on `:5002`). The Worker proxies there when `EXTERNAL_BACKEND` is set.

Conceptual model

```
flowchart LR
    DEV[Vite :3000]
    STATIC[axis_pwa_server :8081]
    FLASK[Flask :5002]
    WRK[Worker :8787]
    CF["(CF Pages + Worker)"]
    DEV --> FLASK
    STATIC --> FLASK
    STATIC --> WRK
    WRK --> FLASK
    CF --> FLASK
```

Topology cheat sheet

Mode	Command sketch	Ports
Dev AXIS	<code>cd frontend && bun run dev</code>	Vite (often <code>:3000</code>)
Dev + Flask	+ <code>make run</code>	<code>:5002</code>
Static dist	<code>bun run build</code> + <code>python axis_pwa_server.py</code>	<code>:8081</code>
Worker	<code>cd frontend/worker && npm run dev</code>	<code>:8787</code>
CF prod	Pages <code>pynescrypt-superchart</code> + Worker same name family	HTTPS



Page map

Page	Topic
Local dev	Day-to-day loop
Build and serve	Vite build, SPA server, assets
Cloudflare	Pages + Worker deploy
VPS demo	Single-box static + Flask
CORS and origins	Allowed origins matrix
CI and testing	GitHub Actions, coverage, e2e

Frozen names

Surface	Value
CF project	<code>pynescrypt-superchart</code>
Health service	<code>pynescrypt-axis-worker</code>

See also

- [Worker](#)
- [Testing reference](#)
- [AXIS hub](#)



LOCAL DEVELOPMENT

Day-to-day AXIS loop: Vite, Flask, Worker wrangler, endpoints, and plugin examples.

LOCAL DEVELOPMENT

Abstract

Local AXIS development is a **multi-process** topology. AXIS is a Solid app under Vite; evaluation usually hits Flask; optional cloud/storage/stream features hit the Worker.

Conceptual model

```
flowchart TB
    subgraph Browser
        UI[Vite HMR app]
    end
    UI -->|engine server| FLASK[Flask :5002]
    UI -->|cloud / run proxy| WRK[wrangler :8787]
    WRK --> FLASK
    UI -->|optional direct| VENUE[Venue REST/WS]
```

Interface surface — commands

Frontend (preferred product path)

```
cd frontend
bun install
bun run dev          # Vite + Solid → typically http://127.0.0.1:3000
```

Flask Pro API (engine backend)

From repo root:

```
make run          # python -m backend.app → :5002
# or hatch / documented backend entry
```

Set PWA endpoint to `http://127.0.0.1:5002` for the **server** engine.

Worker

```
cd frontend/worker
npm install          # or bun
npm run dev          # http://127.0.0.1:8787
```

Point cloud storage / Worker endpoint at `http://127.0.0.1:8787`. For `/api/run` proxy, set Worker `EXTERNAL_BACKEND=http://127.0.0.1:5002` (works on local wrangler; not on CF edge).



Makefile helpers

Target	Intent
<code>make run-frontend</code>	Historical Bun static server (<code>frontend/server.ts</code>) on <code>:8081</code> — legacy shell path
<code>make worker-dev</code>	Wrangler dev
<code>make worker-install</code>	Install worker deps
<code>make test-frontend</code>	Bun unit tests under <code>frontend/tests/</code>

For the **Solid** product, prefer `bun run dev` / `bun run build` over the legacy `server.ts` unless you are debugging the old tree ([legacy shell](#)).

Internals — useful paths

Path	Role
<code>frontend/src/index.tsx</code>	App entry
<code>frontend/vite.config.*</code>	Vite / Solid plugin
<code>frontend/src/engines/catalog.ts</code>	Default engine endpoint
<code>frontend/worker/wrangler.toml</code>	Local vars
<code>frontend/public/plugins/</code>	Example modules served at <code>/plugins/</code>

Worked loop

1. Terminal A: `make run` (Flask).
2. Terminal B: `cd frontend && bun run dev`.
3. Open Vite URL; set engine **Server-Side**, endpoint `http://127.0.0.1:5002`.
4. Load `mock-walk` if venues are blocked.
5. Optional Terminal C: Worker with `ALLOW_OPEN_KEYS=1` for cloud library experiments.

Load an example plugin

With Vite or static server:

```
http://127.0.0.1:3000/plugins/example-coingecko-source.js
```

Manager → Plugins → Load from URL.

Invariants & edge cases

1. **CORS** — Flask must allow the Vite origin; Worker `pickOrigin` allows `:3000` and `:8081`.
2. **Pyodide local** — needs vendor wheels under `public/` (copied to dist on build).
3. **Two endpoints** — topbar endpoint for engine vs cloud storage plugin config can differ (Flask vs Worker).
4. **Legacy Makefile message** still mentions SuperChart Lite and `:8081` Bun server.



Failure modes

Issue	Fix
Engine not ready	Flask down / wrong port
CORS errors	Align origins; see CORS
Plugin 404	File only under <code>src/plugins/</code> not <code>public/plugins/</code>
Worker scripts 401	Set open keys or mint <code>pn_</code> key

See also

- [Build and serve](#)
- [Worker local](#)
- [Plugin examples](#)



BUILD AND SERVE

Vite production build, dist layout, axis_pwa_server.py SPA rules, and asset caching.

BUILD AND SERVE

Abstract

Production AXIS assets are a **static site** produced by Vite (`bun run build` → `frontend/dist`). Serve them with any static host, Cloudflare Pages, or the repo's `axis_pwa_server.py` SPA-aware server.

Critical requirement: **real binary assets** for Pyodide (`.wasm` , `.whl` , `.zip`) must not be rewritten to `index.html` .

Conceptual model

```
flowchart LR
    SRC[src/** Solid]
    PUB[public/**]
    SRC --> VITE[vite build]
    PUB --> VITE
    VITE --> DIST[dist/]
    DIST --> PAGES[CF Pages]
    DIST --> PWA[axis_pwa_server.py]
    DIST --> PREV[vite preview]
```

Interface surface

package.json scripts

```
"dev": "vite",
"build": "vite build",
"preview": "vite preview",
"test": "bun test tests/ worker/tests/",
"test:coverage:gate": "bun run test:coverage && bun scripts/check-coverage.mjs 95",
"test:security": "bun test tests/security/",
"test:e2e:smoke": "playwright test --grep @smoke",
"test:all": "bun run test:coverage:gate && bun run test:security"
```

axis_pwa_server.py

```
cd frontend
bun run build
python axis_pwa_server.py
# HOST / PORT env; default 0.0.0.0:8081 → dist/
```

Behavior (`frontend/axis_pwa_server.py`):



Rule	Detail
Root	Serves <code>frontend/dist</code>
Cache	<code>/assets/*</code> → <code>immutable</code> long cache; else <code>no-cache</code>
SPA fallback	Unknown paths → <code>/index.html</code>
Never rewrite	<code>/assets/</code> , <code>/plugins/</code> , <code>/vendor/</code> , <code>/pyodide/</code> , or extensions including <code>.js/.css/.whl/.wasm/.zip/.py/.json...</code>

This SPA exception list exists because Pyodide + micropip die with opaque `BadZipFile` when HTML is returned for a wheel URL. The engine's `assertZipAsset` detects that class of failure early.

Expected public assets

Path (public → dist)	Purpose
<code>/plugins/*.js</code>	Example dynamic plugins
<code>/vendor/*.whl</code>	pynescrypt + antlr wheels
<code>/pyodide/v0.26.2/**</code>	Self-hosted Pyodide
<code>/pyodide/pynescrypt_runtime.py</code>	Browser run bridge

Internals

Path	Role
<code>frontend/package.json</code>	Build scripts
<code>frontend/axis_pwa_server.py</code>	Threading HTTP SPA server
<code>frontend/src/engines/catalog.ts</code>	Asset URL expectations
<code>frontend/LEGACY.md</code>	Product path vs old shell

Makefile note

`make pages-deploy` may historically deploy the wrong tree — **prefer**:

```
cd frontend && bun run build
wrangler pages deploy dist --project-name=pynescrypt-superchart
```

Worked example — static desk demo

```
cd frontend
bun install
bun run build
PORT=8081 python axis_pwa_server.py
# another terminal
make run    # Flask :5002
```

Open `http://127.0.0.1:8081`, engine endpoint `http://127.0.0.1:5002`.



Invariants & edge cases

1. **Service workers / offline** — product path uses Vite PWA assets; legacy root `sw.js` is not the shipping SW.
2. **Relative pyodide index** — resolves against `location.origin`.
3. **Do not hand-edit dist** — regenerate from build.

Failure modes

Symptom	Cause
Blank routes on refresh	Server lacks SPA fallback
Pyodide HTML wheel error	Fallback rewriting <code>.whl</code> or missing vendor
Missing plugins	<code>public/plugins</code> not copied (check Vite publicDir)

See also

- [Local dev](#)
- [Cloudflare](#)
- [Engines](#)



CLOUDFLARE DEPLOYMENT

Pages + Worker for AXIS: frozen project pyonescript-superchart, bindings, deploy order, EXTERNAL_BACKEND reality.

CLOUDFLARE DEPLOYMENT

Abstract

Production AXIS on Cloudflare is a **split deploy**:

Piece	Role
Pages	Static PWA (frontend/dist)
Worker	JSON API + WebSocket (frontend/worker)

Project id: pyonescript-superchart — frozen infrastructure name (dashboard, wrangler, CI). Do not rename without a full binding/domain migration.

Evaluation truth: Worker /api/run proxies to EXTERNAL_BACKEND unless the gated Pyodide path is fully implemented and enabled. You still need a Python host (VPS Flask, container, etc.) for real Pine fidelity today.

Conceptual model

```
flowchart TB
    USER[Browser]
    USER --> PAGES[CF Pages dist]
    USER --> WRK[CF Worker]
    WRK -->|EXTERNAL_BACKEND| VPS[Flask / PYNE host]
    WRK --> KV[KV]
    WRK --> D1[D1]
    WRK --> DO[SessionDO]
```

Deploy procedure

1. Provision bindings (once)

See [Worker bindings](#):

- KV API_KEYS , USAGE
- D1 pyonescript + schema
- Optional R2, Durable Objects

2. Configure secrets / vars

Production checklist:



Setting	Production guidance
EXTERNAL_BACKEND	Public HTTPS URL of Flask/PYNE API
ALLOWED_ORIGIN	Exact Pages origin (e.g. <code>https://...pages.dev</code> or custom)
ADMIN_TOKEN	Secret, strong
ALLOW_OPEN_KEYS	<code>"0"</code> or unset
PYODIDE_IN_WORKER	<code>"disabled"</code> until wheel pipeline ready

3. Deploy Worker

```
cd frontend/worker
npm install
npx wrangler deploy
```

4. Build & deploy Pages

```
cd frontend
bun install
bun run build
npx wrangler pages deploy dist --project-name=pynescrypt-superchart
```

Worker package also defines:

```
"deploy:pages": "wrangler pages deploy ../dist --project-name=pynescrypt-superchart"
```

5. Point the PWA

- Engine endpoint: Worker URL (if path-mapped to `/run`) or Flask URL directly
- Cloud storage endpoint: Worker origin
- Ensure CORS allows Pages origin

Internals

Path	Role
frontend/worker/wrangler.toml	Worker name + bindings
frontend/worker/README.md	Ops narrative
frontend/worker/package.json	deploy scripts
Makefile worker-deploy / pages-deploy	Convenience (verify Pages path)

Health check

```
curl -sS https://<worker>/health
# service: pynescrypt-axis-worker
```



Invariants & edge cases

1. **Pages $\neq$ Worker host** — CORS required unless same-origin routing via custom domain routes.
2. **Localhost EXTERNAL_BACKEND** is useless from the edge.
3. **DO migrations** apply on deploy when uncommented.
4. **Observability** enabled in wrangler for log tails (`wrangler tail`).

Failure modes

Issue	Fix
503 NO_BACKEND	Set reachable EXTERNAL_BACKEND
CORS blocked	ALLOWED_ORIGIN mismatch
SPA HTML for wheels	Pages must serve <code>/vendor</code> and <code>/pyodide</code> as static files
Stream NO_DO	Enable SessionDO bindings

See also

- [Worker](#)
- [CORS](#)
- [VPS demo](#) (backend co-host)



VPS DEMO TOPOLOGY

Single-box demo: static AXIS dist + Flask Pro API (+ optional reverse proxy) without Cloudflare.

VPS DEMO TOPOLOGY

Abstract

A **VPS demo** is the lowest-surprise production-like setup for Pine evaluation: one machine runs Flask (PYNE) and serves the static AXIS build. Optional nginx/caddy terminates TLS and reverse-proxies `/run` same-origin to avoid CORS.

No Worker required. Cloud storage and DO streams will not work without the Worker.

Conceptual model

```
flowchart LR
    U[Users]
    U --> NGX[nginx/caddy :443]
    NGX --> STATIC[dist]
    NGX --> FLASK[Flask :5002]
```

Interface surface

Processes

Process	Command	Port
Build (CI or once)	<code>cd frontend && bun run build</code>	—
Static	<code>PORT=8081 python axis_pwa_server.py</code>	8081
API	<code>make run / python -m backend.app</code>	5002

Same-origin proxy sketch (Caddy)

```
axis.example.com {
  handle /run* {
    reverse_proxy 127.0.0.1:5002
  }
  handle {
    reverse_proxy 127.0.0.1:8081
  }
}
```

Then PWA endpoint = `https://axis.example.com` and the **server** engine posts to `/run` on the same host.

Minimal without TLS

- Static `:8081`, Flask `:5002`



- Set endpoint `http://VPS_IP:5002`
- Open firewall; accept CORS from static origin (configure Flask accordingly)

Internals

Concern	Notes
Pyodide offline	Ship full <code>dist</code> including <code>vendor</code> + <code>pyodide</code>
Process supervisor	systemd units for flask + <code>axis_pwa_server</code>
Updates	Rebuild dist artifact; restart static only
Secrets	Flask admin / pro keys separate from Worker

Worked example — systemd sketch

```
# /etc/systemd/system/axis-pwa.service
[Service]
WorkingDirectory=/opt/axis/frontend
Environment=PORT=8081
ExecStart=/usr/bin/python3 axis_pwa_server.py
Restart=on-failure
```

Pair with an existing `backend` unit for Flask.

Invariants & edge cases

1. **CPU/RAM** — Pyodide is browser-side; VPS load is Flask evaluate concurrency.
2. **Do not expose open admin tokens.**
3. **Disk** — dist + wheels are tens of MB.
4. Hybrid: VPS Flask as `EXTERNAL_BACKEND` for a CF Worker still works if VPS has a public URL.

Failure modes

Issue	Fix
Mixed content	HTTPS page calling HTTP Flask blocked
CORS	Prefer same-origin proxy
502 on /run	Flask crashed; check <code>journalctl</code>

See also

- [Build and serve](#)
- [Cloudflare](#)
- [CORS](#)



CORS AND ORIGINS

How AXIS Worker pickOrigin works, local allowlist, ALLOWED_ORIGIN, and Flask/browser constraints.

CORS AND ORIGINS

Abstract

AXIS is a browser product that talks to **cross-origin** backends (Flask, Worker, venues). CORS misconfiguration is the #1 local-deploy footgun after missing wheels.

The Worker implements an explicit origin picker; Flask and venue APIs have their own rules.

Conceptual model

```
flowchart TD
    ORIGIN[Browser Origin header] --> W[Worker pickOrigin]
    W -->|localhost:3000/8081| ECHO[Echo request origin]
    W -->|else| ALLOWED[env.ALLOWED_ORIGIN]
    ORIGIN --> F[Flask CORS middleware]
    ORIGIN --> V[Venue APIs]
```

Worker behavior

From `frontend/worker/src/index.ts`:

Always echoed (dev allowlist):

- `http://localhost:8081`
- `http://127.0.0.1:8081`
- `http://localhost:3000`
- `http://127.0.0.1:3000`

Otherwise: `env.ALLOWED_ORIGIN` or default `https://pynescrypt.ai`.

CORS headers applied:

```
Access-Control-Allow-Origin: <picked>
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization, X-Admin-Token, If-Match
Access-Control-Max-Age: 86400
Vary: Origin
```

`OPTIONS` → `204` preflight.

Scripts handler duplicates allow headers for consistency.

Production implication

If Pages is `https://pynescrypt-superchart.pages.dev` (example), set:



```
ALLOWED_ORIGIN = "https://pynescrypt-superchart.pages.dev"
```

or your custom domain. **Exact string match** — no wildcard logic in code.

Flask / server engine

The **server** engine posts from the browser to the configured endpoint. Flask Pro API must:

1. Answer `OPTIONS` preflight if cross-origin.
2. Reflect or allow the PWA origin.
3. Accept `Content-Type: application/json`.

Same-origin reverse proxy ([VPS demo](#)) eliminates CORS for `/run`.

Venue sources/streams

Public Binance/OKX/etc. REST must send CORS headers usable by browsers. If blocked:

1. Use offline `mock-walk` / `mock-poll`.
2. Add a **same-origin proxy** on Worker/Flask (not a general open proxy — scope carefully).
3. Prefer DO relay only for WS fan-out (WS is not CORS in the XHR sense, but still network-reachable).

Dynamic plugins

- **Module import** of plugin JS: needs CORS on the **script origin** (same-origin `/plugins/` is safe).
- **fetch inside plugin**: subject to third-party CORS independently.

Invariants & edge cases

1. **Credentials** — Worker CORS does not set `Allow-Credentials: true`; use Bearer headers, not cookies.
2. **Multiple prod domains** — code picks a single `ALLOWED_ORIGIN`; multi-domain needs code change or edge rewrite.
3. **Vite vs 8081** — both covered for Worker; Flask config must match whichever you use.
4. **Health checks** from curl omit Origin — still work; browser calls need correct ACAO.

Failure modes

Browser console	Meaning
blocked by CORS policy	Origin not allowlisted
preflight 404	Server lacks OPTIONS
No ACAO on error body	Some error paths forgot headers (Worker try/catch mostly covered)

See also

- [Worker bindings](#)
- [Local dev](#)



- [Dynamic loader](#)



CI AND TESTING

GitHub Actions for AXIS: unit coverage gate, Playwright smoke, worker typecheck, nightly full e2e.

CI AND TESTING

Abstract

AXIS quality gates live primarily under `frontend/` (Bun tests) and `frontend/worker/tests/`, orchestrated by GitHub Actions (`.github/workflows/ci.yml`, `axis-nightly.yml`). Deep test conventions are documented in [Testing reference](#) and `frontend/TESTING.md`.

Conceptual model

```
flowchart TB
    PR[Pull request] --> UNIT[Bun unit + worker tests]
    PR --> GATE[Coverage gate 95%]
    PR --> E2E[Playwright @smoke]
    PR --> WTC[Worker tsc]
    NIGHT[Nightly cron] --> FULL[Playwright all]
    NIGHT --> SEC[Security + coverage]
```

Interface surface — local commands

```
cd frontend
bun run test                # tests/ + worker/tests/
bun run test:unit
bun run test:coverage:gate  # lcov + scripts/check-coverage.mjs 95
bun run test:security
bun run test:e2e:smoke
bun run test:e2e:critical
bun run test:e2e
bun run test:all            # gate + security
```

Worker:

```
cd frontend/worker
npm run typecheck
```

Repo Makefile:

```
make test-frontend
make worker-typecheck
```



CI workflows

ci.yml (excerpted jobs)

Job (names vary)	What
Frontend tests / coverage	Bun install, coverage, artifact <code>axis-coverage-lcov</code>
<code>axis-e2e</code>	Playwright <code>@smoke</code> on PR
Worker typecheck	<code>frontend/worker</code> tsc
Related	Python suite remains for pynescrypt core

axis-nightly.yml

Job	What
Playwright full	all e2e specs
Security + coverage gate	<code>test:coverage:gate</code> + <code>test:security</code>

Schedule: `0 4 * * *` UTC + `workflow_dispatch`.

Internals

Path	Role
<code>frontend/TESTING.md</code>	Authoritative testing guide
<code>frontend/scripts/check-coverage.mjs</code>	Scoped line gate
<code>frontend/playwright.config.ts</code>	webServer build+preview <code>:4173</code>
<code>frontend/e2e/</code>	Specs with <code>@smoke</code> / <code>@critical</code> tags
<code>frontend/worker/tests/</code>	auth, keys, scripts, runtime
<code>.github/workflows/ci.yml</code>	PR gates
<code>.github/workflows/axis-nightly.yml</code>	Nightly

Coverage policy (summary)

- **Gate:** 95% lines on a **scoped** core (plugins, storage minus idb, store, results, sources, streams, data, selected chart helpers, worker auth/keys/runtime/scripts).
- **Excluded from gate:** heavy chart UI, pyodide boot, legacy JS, Solid `.tsx` surfaces (unit-tested elsewhere or deferred).

E2E design

- Builds production preview; baseURL `http://127.0.0.1:4173`.
- Mocks `/run` and Binance where possible so smoke does not need live Pro API.
- Selectors use `data-testid` (`axis-btn-load`, `axis-manager`, ...).



Invariants & edge cases

1. **No real network in unit tests** — mock `fetch` / `WebSocket`.
2. **Import `./setup` first** when touching store/plugins.
3. **CI Bun version** pinned in workflows (e.g. 1.3).
4. **Playwright browsers** installed with `--with-deps` on CI.

Failure modes

CI red	Likely
Coverage gate	New untested code in scoped paths
Smoke timeout	Build slow; webServer 180s
Worker tsc	Env typing / DO exports
Flaky e2e	Missing testid; race without mock

See also

- [Testing reference](#)
- [Local dev](#)